

M13 — Dual chart mode (private_clone / shared)

Phase: 8 — Chart Mode **Status:** DONE (2026-05-26) **Blocked by:** M11 **Blocks:** M14 **Estimated effort:** 2.5 days · **Actual:** 0.5 day

1. Purpose

Two pedagogical chart modes for a v7 encounter:

- **shared (v6 default)** — every student assigned to an encounter

documents into the SAME chart. Multiple bedside chat stations share one `chart_event` log. This is the v6 behavior and the M10 byte-for-byte compat contract.

- **private_clone (new)** — the encounter is a TEMPLATE; each

student who joins gets their OWN clone of the template (new encounter id, new join code, fresh chart). N students = N independent encounters all seeded from the same template, all starting with the same chart, then diverging as students chart independently.

Use cases:

- **Shared:* clinical-handoff exercises where multiple students

coordinate care on one patient (charge-nurse + bedside RN + family).

- **Private clone:* skills practice where every student should run

the same case independently and get their own debrief.

The instructor picks per-encounter at wizard time (Step 4r row field or Activity catalog `default_chart_mode`).

2. Structure

Files touched:

- `portal/control_session.py` — adds `cloned_from_id: str | None`

to `ControlSession`. Templates have `cloned_from_id=None`; clones point at their template's id.

- `portal/control_room.py` — adds three methods on `ControlRoom`:

- `clone_encounter(template_id, label_suffix="")` — creates a

fresh `ControlSession` cloning the template's `scenario_name`, `scenario_text`, `modules`, `persona`, `ehr`, `chart_mode`, `activity_id`; gets a new `id` and `join_code`; carries `cloned_from_id`; registers itself in `room.encounters`. - `is_template(encounter_id)` — True iff `chart_mode='private_clone'` AND `cloned_from_id` is `None`. - `encounters_for_join_picker()` — list of encounters shown on the M9 student-join page: shared encounters + `private_clone` templates; clones are filtered out (each clone belongs to one student already).

- `portal/server.py` — `portal_students_register` (M9 handler):

- Before assigning the student, check if the picked encounter is

a `private_clone` template. If yes, call `room.clone_encounter()` +

`_ensure_ehr_session_registered(clone)` and use the CLONE as the target. The student is assigned to the clone, not the template. - Response carries `is_clone` and `cloned_from_id` fields so the client knows

the redirect points at a clone. - The M9 `/portal/students/join` GET handler now uses `room.encounters_for_join_picker()` instead of the raw `room.encounters` dict, hiding clones from the picker.

No new files — M13 extends the existing M9 wiring.

3. Uses

Bedside student flow (private_clone): 1. Student scans room QR → lands on `/portal/students/join?code=...` 2. Picker shows the template "Bed 1 — Diaz (template)" alongside any shared beds. 3. Student picks the template + types their name. 4. Server: clone the template → assign student to the clone → register a fresh `ehr_session` for the clone (seed is deterministic on persona, so every clone starts identically) → create chat station on the clone → redirect to `/station/<clone_join_code>/<station_id>`. 5. The next student picking the same template gets a different clone.

Bedside student flow (shared): identical to M9 — the picker shows the encounter, the student is assigned to it, gets a chat station on it, lands on `/station/<encounter_join_code>/...` The encounter's `assigned_student_ids` list grows; no clone is made.

Instructor dashboard: `/api/room/state` lists the template AND every clone, so the instructor sees what each student is working on. The student-join picker hides clones; the dashboard does not.

4. Functions (exported API surface)

ControlSession dataclass (extended)

New field	Type	Default	Purpose	
<code>cloned_from_id</code>	<code>`str`</code>	<code>None`</code>	None	Set on clones to the template's encounter id. None on templates and shared-mode encounters.

ControlRoom methods (new)

Symbol	Signature	Purpose
<code>clone_encounter</code>	<code>(template_id, *, label_suffix="") -> Encounter</code>	Build a fresh per-student clone. Inherits scenario content; gets new id + join code. Adds the clone to <code>room.encounters</code> . Raises <code>KeyError</code> on unknown <code>template_id</code> .
<code>is_template</code>	<code>(encounter_id) -> bool</code>	True if encounter exists, is <code>chart_mode='private_clone'</code> , and has no <code>cloned_from_id</code> .
<code>encounters_for_join_picker</code>	<code>() -> list[Encounter]</code>	Visible-to-student encounter list. Shared + templates included; clones hidden.

/portal/students/register response shape (extended)

New field	Purpose
<code>is_clone</code>	True when the student joined a private clone (template was cloned server-side).

New field	Purpose
<code>cloned_from_id</code>	The template's encounter id, when <code>is_clone</code> is True.

The student-facing JS doesn't need to know about clones — it just follows the `redirect_url`. The new fields exist for testing, debugging, and a future "you're working in a private session" indicator on the bedside chat station.

5. Limitations

- **Templates accumulate no chart events from the student-driven**

flow — students always work on clones, never on the template. However, ***room-level scene broadcasts*** (M4/M7) currently target every encounter including templates, so a "scene_broadcast all" would write a row to the template too. This is wasteful but harmless (the template has no students viewing it). A future enhancement could exclude templates from broadcasts; deferred.

- **A clone's seed is deterministic on persona id** — same as a

v6 single-patient session seeded from the same persona. Two clones of the same template start with IDENTICAL `chart_event` state (typically empty before any scene fires). They diverge the moment a student or instructor writes the first event.

- **Clones cannot be re-cloned.** A clone has

`chart_mode='private_clone'` but `cloned_from_id` is set; the `is_template()` check returns False; the join picker hides it; the register handler will not clone it again. This is intentional — clones belong to one student and shouldn't be a template for further clones.

- **No instructor "Reassign student to different clone" affordance**

M13. If a student leaves mid-sim, their clone is orphaned; another student joining the template gets a fresh clone. The orphaned clone keeps its `chart_event` log for debrief. A v7.1 cleanup module could prune them.

- **Capacity hardening (M19) needs to count clones against the**

10-encounter cap. A `private_clone` template with 8 students generates 9 encounters in the room (1 template + 8 clones). M19 should treat the cap as "encounters total," not "templates."

- **Cohort debrief (M14) sees the same `chart_event` API for every**

clone — it'll naturally produce per-student debrief facets for `private_clone` rooms.

- **`scenario_name` and `encounter_label` on the clone include a**

``(<student name>)` suffix` to make the dashboard list scannable ("Bed 1 — Diaz (template)" → "Bed 1 — Diaz (Alice)" / "Bed 1 — Diaz (Bob)"). The bedside chat title shows the suffix too — operator should be aware.

6. Test status

Automated (tests/v7/test_private_clone_*.py, test_shared_mode_*.py)

Test file	Cases	Status	Last run
test_private_clone_creates_n_encounters_for_n_students.py	2 — 3 students each get their own clone (4 encounters total = 1 template + 3 clones); join-page picker still shows only the template (clones hidden).	PASS	2026-05-26
test_shared_mode_single_encounter_for_all_students.py	2 — shared bed has exactly 1 encounter with 3 assigned students after 3 joins; mixed shared + private in one room behaves per-encounter (shared keeps 1, private spawns clones).	PASS	2026-05-26
test_private_clone_charts_are_isolated.py	3 — chart events written to clone A do not appear in clone B or in the template; clones inherit scenario content; instructor dashboard lists clones alongside templates.	PASS	2026-05-26

7/7 PASS. **Full v7 suite: 91/91 passing** (up from 84 — +7 M13 tests). **Full v6 regression on v7: 202 passed**, same 6 env-flaky pre-existing failures, **0 v7 regressions**.

Manual

Browser preview verification deferred — the data contract is covered end-to-end by the tests (template → clone → station redirect). The visible UX change (clones appear on the dashboard but not on the student picker) is exercised by `test_dashboard_state_lists_clones_alongside_templates` and `test_private_clone_template_is_filtered_from_join_picker_after_clones_exist`.

7. Change list

Date	Author	Change	Files
2026-05-26	claude-code	Added <code>cloned_from_id</code> to <code>ControlSession</code> . Three new <code>ControlRoom</code> methods (<code>clone_encounter</code> , <code>is_template</code> , <code>encounters_for_join_picker</code>). M9 register handler now clones templates and assigns the student to the clone; response carries <code>is_clone</code> + <code>cloned_from_id</code> . M9 GET join page uses the picker filter so clones are hidden. 7 acceptance tests across 3 files.	<code>portal/control_session.py</code> , <code>portal/control_room.py</code> , <code>portal/server.py</code> , <code>tests/v7/test_private_clone_creates_n_encounters_for_n_students.py</code> , <code>tests/v7/test_shared_mode_single_encounter_for_all_students.py</code> , <code>tests/v7/test_private_clone_charts_are_isolated.py</code>

8. Open questions / known issues

- **Scene broadcasts hit templates.** M4's `scene_broadcast` with

`targets="all"` iterates every encounter in `room.encounters`, including templates. The template has no students, so the written rows aren't visible to anyone — wasteful but harmless. Fix: filter templates out of broadcast targets. ~10 lines in `_apply_scene` plumbing. Deferred until operator surfaces it in feedback.

- **Cohort debrief (M14) per-student grouping.** In `private_clone`

mode each clone is a separate `session_id` with one student. M14 should aggregate at the room level so the cohort debrief shows "every student's run side-by-side" cleanly. The data is already shaped right for that — `student_id` → `encounter_id` → `chart_event` — but the rendering UI in M15 needs to know about the template/clone relationship.

- **Capacity hardening (M19) cap math.** The 10-encounter cap

needs to include clones. Document in M19's spec.

- **Voice budgets and the per-room Haiku cap (M17)** also need to

treat clones independently — each clone runs its own LLM conversation. M17's per-encounter cap math should apply per clone, not per template.

- **Dashboard UI grouping (M5 follow-up).** The `/api/room/state`

payload now includes `chart_mode` and (via the underlying encounter) `cloned_from_id`. A future dashboard pass should visually group clones under their template card.